EECS3311 Software Design

Course Project: Design, Build, and Test an AI Agent

1. Project Overview

In this project, you will **design, implement, and systematically test an AI agent-based software system**.

Unlike a traditional software project, the goal is not only to produce working code. You are expected to apply the software design principles learned in EECS3311 to build a system that combines:

- traditional deterministic software components;
- one or more Large Language Models (LLMs);
- agent capabilities such as reasoning, planning, memory, retrieval, and tool use;
- external APIs, tools, databases, or other resources where appropriate.

Use of AI development tools is required for this project. You are expected to collaborate with AI during implementation rather than simply writing the entire system manually.

The project consists of three stages:

Stage 1 — Design: What are you going to build, and how will the software be designed?

Stage 2 — AI-Human Collaborative Implementation: How will you implement the design with the assistance of AI development tools?

Stage 3 — Testing and Validation: Does the deterministic software work correctly, and does the AI agent behave reliably?

A major goal of the project is to establish traceability across these stages:

Requirements → Use Cases → Design → Classes → Implementation → Tests

Stage 1 — Software and Agent Design

2. Objective

Before implementation, you must develop a clear software design for your agent.

Your design should answer four fundamental questions:

1. **What does your agent do?**
2. **How does a user interact with it?**
3. **What components/classes are required?**
4. **How do these components collaborate to implement each feature?**

Do not start by asking an LLM to generate an entire application. You should first establish the architecture and major design decisions.

3. Project Description and Scope

Provide a concise description of your proposed agent.

Your description should include:

3.1 Problem

What problem does your agent solve?

Explain:

- who the intended users are;
- what problem they currently face;
- why an agent is useful for solving this problem.

3.2 Main Features

Identify the major features of your system.

For example, an agent might provide:

- natural-language interaction;
- information retrieval;
- task planning;

- tool/API execution;
- document processing;
- code generation;
- recommendations;
- memory of previous interactions;
- autonomous execution of multi-step tasks.

For each feature, briefly explain what it does and why it is needed.

Your project should contain enough functionality to demonstrate meaningful software design. A simple chatbot that only sends a prompt to an LLM API and displays the response is **not sufficient**.

4. Use Cases

Identify the major use cases of your system.

For each important use case, provide:

- Use Case ID;
- Use Case Name;
- Primary Actor;
- Goal;
- Preconditions;
- Trigger;
- Main Success Scenario;
- Alternative/Exception Flows;
- Postconditions.

For example:

UC-01: Ask the Research Agent to Find Papers

Actor: User

Precondition: The system is running and has access to the search tool.

Main Flow:

1. User enters a research question.
2. Agent interprets the request.
3. Agent creates a search query.
4. Agent invokes the search tool.
5. Search results are returned.
6. Agent evaluates the results.
7. Agent generates a response.
8. System presents the response to the user.

Alternative Flow:
If the search tool fails, the agent informs the user and either retries or provides an alternative.

Include a **UML use-case diagram** showing the major actors and use cases.

5. Sequence Diagrams

Create sequence diagrams for the major use cases.

The diagrams should show how the major software components collaborate during execution.

For example:

User → UI → AgentController → Planner → LLMClient → ToolManager → SearchTool

A sequence diagram should show:

- participating objects/components;
- messages between components;
- order of interactions;
- important return values;
- alternative/error flows where appropriate.

Do not represent the entire application as:

User → Agent → LLM

The purpose of the diagram is to expose the **software design behind the agent**.

6. Class Diagram

Develop a UML class diagram representing the architecture of your system.

The diagram should include the major:

- classes;
- interfaces;
- important attributes;
- important methods;
- associations;
- dependencies;
- inheritance relationships;
- aggregation/composition relationships where appropriate.

For example, an agent system might contain classes such as:

AgentController

Planner

MemoryManager

ToolManager

LLMClient

PromptBuilder

ResponseParser

SearchTool

FileTool

ConversationHistory

Your actual classes should reflect the needs of your own project.

You are expected to demonstrate appropriate software-design principles, including where relevant:

- abstraction;
- encapsulation;
- separation of concerns;
- high cohesion;
- low coupling;
- interfaces;
- dependency inversion;
- polymorphism;
- appropriate design patterns.

7. Feature-to-Design Mapping

This is an important part of Stage 1.

For **each major feature**, explain how it will be implemented using your proposed classes.

For example:

Feature	**Main Classes**	**Responsibilities**
User interaction	ChatUI, AgentController	Receive requests and display results
Planning	Planner, LLMClient	Decompose a request into steps
Tool execution	ToolManager, Tool	Select and execute tools
Web search	SearchTool	Perform external searches
Memory	MemoryManager	Store and retrieve relevant history
LLM interaction	LLMClient, PromptBuilder	Construct and execute LLM requests

You should be able to explain:

For Feature X, which classes are involved, what is each class responsible for, and how do these classes collaborate to implement the feature?

The implementation in Stage 2 should be consistent with this design. Significant deviations must be documented and justified.

Stage 2 — AI-Human Collaborative Implementation

8. Objective

In Stage 2, you will implement your design.

Using AI tools is mandatory.

Modern software development increasingly involves collaboration between developers and AI coding tools. An important learning objective of this project is therefore to understand how software design guides AI-assisted development.

You remain responsible for the final software.

AI-generated code should not simply be accepted because it compiles or appears reasonable. You should understand, inspect, integrate, and test the generated implementation.

9. AI Tools and Models

Document all major AI tools used during development.

Examples include AI coding assistants, IDE agents, command-line coding agents, or general-purpose LLM interfaces.

For each tool, report:

Tool	LLM Model	Purpose	Example Tasks
Tool A	Model X	Code generation	Implemented tool interface
Tool B	Model Y	Debugging	Diagnosed API failure
Tool C	Model Z	Refactoring	Refactored agent controller

You must explicitly report, when available:

- AI tool/platform;
- LLM/model used;
- purpose of using the tool;
- major parts of the project where it was used.

If the tool automatically chooses models and does not expose the exact model, state this clearly rather than guessing.

10. AI-Human Collaboration Log

Maintain a lightweight record of important AI interactions during development.

You do **not** need to submit every prompt.

Instead, select representative interactions that demonstrate how AI contributed to your development process.

For each selected interaction, record:

Task: What were you trying to accomplish?

Input/Instruction: What did you ask the AI to do?

AI Contribution: What did the AI generate, suggest, modify, or diagnose?

Human Contribution: What did you review, correct, redesign, reject, or improve?

Outcome: What was eventually incorporated into the project?

This should demonstrate genuine **AI-human collaboration**, rather than simply stating, "We used ChatGPT."

11. Design-to-Implementation Traceability

Your implementation should reflect your Stage 1 design.

For every major feature, identify the classes/files that implement it.

For example:

Feature	Designed Classes	Implemented Classes/Files	Status
Planning	Planner	Planner.java	Implemented
Search	SearchTool	SearchTool.java	Implemented
Memory	MemoryManager	MemoryManager.java	Modified
Tool management	ToolManager	ToolManager.java	Implemented

If the implementation differs from your original design, explain:

Original Design → Change → Reason for Change

Changing the design during implementation is acceptable. Good software design evolves. However, changes should be intentional and justified.

Stage 3 — Testing and Agent Validation

12. Objective

Testing an AI agent is different from testing a traditional deterministic program.

Your system contains two broad types of behavior:

Deterministic behavior

Given the same input and program state, the component should normally produce a predictable result.

Examples include:

- parsers;
- data validation;
- database operations;
- calculations;
- API wrappers;
- tool implementations;
- state-management logic;
- formatting;

- deterministic business logic.

Agent/LLM behavior

The result may depend on an LLM and may therefore be non-deterministic.

Examples include:

- reasoning;
- planning;
- tool selection;
- instruction following;
- generated responses;
- recovery from unexpected situations;
- multi-step agent behavior.

These two types of behavior require different testing strategies.

13. Part A — Testing Deterministic Components

Write automated tests for the deterministic components of your application.

Use the appropriate testing framework for your implementation language.

Your tests should include, where appropriate:

- unit tests;
- integration tests;
- normal cases;
- boundary cases;
- invalid inputs;
- exception/error handling;
- interactions between components.

For each important deterministic class, identify what needs to be tested.

Do not test only getters/setters or trivial methods. Focus on meaningful application behavior.

Your submission should include:

- test source code;
- instructions for running the tests;
- number of tests;
- test results;
- a short discussion of important failures found and fixed.

14. Part B — Agent Behavior Testing with KUMA

Traditional assertions such as:

assert output == expectedOutput

are often insufficient for testing an LLM-based agent.

You will therefore use **KUMA (https://github.com/DefuzeX-AI/KUMA-DefuzeX)** to systematically test the behavior of your agent.

Your goal is to determine whether the agent behaves correctly and reliably under different situations, not merely whether the application runs.

15. Define Agent Behavioral Requirements

Before creating agent tests, define the expected behaviors of your agent.

Examples include:

BR-01 — Correct Tool Selection
When a task requires external information, the agent should select an appropriate information-retrieval tool.

BR-02 — Tool Result Usage
The agent should incorporate relevant tool results into its final response.

BR-03 — Invalid Tool Input Handling
The agent should not execute a tool with invalid arguments.

BR-04 — Failure Recovery
If a tool fails, the agent should recover appropriately rather than silently producing an unsupported answer.

BR-05 — Instruction Following
The agent should follow important constraints specified by the user.

BR-06 — Task Completion
The agent should complete the requested task rather than stopping after intermediate reasoning or tool execution.

Your behavioral requirements should be specific to **your own agent**.

16. Create Behavioral Test Cases

For each behavioral requirement, design multiple test cases.

A behavioral test should specify:

Field	Description
Test ID	Unique identifier
Requirement	Behavior being tested
Input	User/task input
Context	Relevant environment/state
Expected Behavior	What the agent should do
Prohibited Behavior	What the agent should not do
Evaluation	How success/failure is determined

For example:

Test: BT-07

Requirement: BR-01 — Correct Tool Selection

Input: "Find recent papers about software testing agents."

Expected Behavior: The agent should invoke the configured literature/search tool before answering.

Prohibited Behavior: The agent should not fabricate papers without using the available retrieval capability.

17. Test Beyond Happy Paths

Your KUMA tests should not consist only of simple tasks that the agent is expected to pass.

Include challenging situations such as:

- ambiguous requests;
- incomplete information;
- unexpected tool outputs;
- invalid inputs;
- unavailable tools;
- conflicting instructions;
- irrelevant retrieved information;
- malformed responses;
- long conversations;
- repeated requests;
- edge cases;
- adversarial or misleading inputs where appropriate.

The objective is to discover **where your agent fails**.

A strong testing report is not one in which every test passes. Finding and understanding meaningful failures is an important outcome.

18. Analyze Agent Failures

For failed KUMA tests, analyze what happened.

Do not report only:

"The agent failed 7 tests."

Instead, investigate the failure behavior.

Possible failure categories include:

- incorrect reasoning/planning;
- wrong tool selection;
- incorrect tool arguments;
- failure to use tool output;
- instruction-following failure;
- hallucinated information;
- incomplete task execution;
- failure recovery problem;
- memory/context problem;
- output-format violation.

For important failures, provide:

Input → Expected Behavior → Observed Behavior → Failure Category → Likely Cause

19. Improve and Retest

Select important failures discovered during testing and attempt to improve your system.

Improvements may involve:

- prompt changes;
- class/design changes;
- better tool descriptions;
- input validation;
- output validation;
- additional deterministic safeguards;
- retry mechanisms;

- improved context management;
- better agent instructions.

Then rerun the relevant tests.

Report:

Before Fix → Modification → After Fix

This demonstrates that testing is being used to improve the software rather than simply to produce a final score.

20. Final Submission

Your final project submission should contain the following major artifacts.

Stage 1 — Design

- Project description and scope
- Major features
- Use cases
- UML use-case diagram
- Sequence diagrams
- UML class diagram
- Feature-to-class/design mapping
- Explanation of important design decisions

Stage 2 — Implementation

- Complete source code
- README and setup instructions
- AI tools used
- LLM models used
- Representative AI-human collaboration log
- Feature-to-implementation mapping

- Explanation of important design changes

Stage 3 — Testing

- Automated tests for deterministic components
- Test execution results
- Agent behavioral requirements
- KUMA behavioral tests
- KUMA execution results
- Failure analysis
- Improvements made based on testing
- Retesting results

21. Lesson Learnt in the Final Project Report

Your final report should tell the complete story of the project:

What did you want to build?

↓

How did you design it?

↓

How did the design map to classes and interactions?

↓

How did you collaborate with AI to implement it?

↓

How did you test the deterministic software?

↓

How did you test the non-deterministic agent behavior?

↓

What failures did you discover?

↓

How did you improve the agent?

The report should focus on **software design, implementation decisions, AI-human collaboration, testing, and lessons learned**, rather than simply providing screenshots of the final application.

22. Important Project Principles

Throughout the project, keep the following principles in mind:

Design before implementation.
Your UML diagrams and design artifacts should guide development rather than being created afterward to match existing code.

AI use is required, but human responsibility remains.
AI can generate code, explain APIs, debug problems, and suggest designs. You are responsible for understanding and validating the resulting system.

Maintain traceability.
A reader should be able to trace an important feature from its requirement and use case to its classes, implementation, and tests.

Separate deterministic testing from agent testing.
Traditional software components should be tested using conventional automated tests. Non-deterministic agent behavior should be evaluated through behavioral testing with KUMA.

Testing should discover problems.
The purpose of testing is not to demonstrate that your agent is perfect. Meaningful failures, careful analysis, and successful improvements are evidence of good software engineering.

The central theme of the project is therefore:

Design it → Build it with AI → Test the software → Test the agent → Learn from failures → Improve it.